

LAB 1: REQUIREMENTS ENGINEERING & UML USE-CASE MODELLING

Problem Statement #44 – Database Query Performance Profiler

Name: Srijan Akshit | SRN: PES1UG24AM284 | Student SI. No.: 45

1. Problem Context

A database observability tool that ingests slow query logs, parses SQL execution plans, identifies missing index candidates, and generates weekly performance optimization digests. Target stakeholders/actors: Database Administrator and Backend Lead.

2. Requirements Table

The table contains exactly 5 Functional Requirements (FR-001 to FR-005) and 2 Non-Functional Requirements (NFR-001 to NFR-002).

ID	Type	Description	Priority	Acceptance Criteria	Rationale
FR-001	Functional	The system shall ingest slow query logs from PostgreSQL/MySQL databases.	High	Pass: Slow query records are successfully ingested and stored for analysis. Fail: Records are lost or rejected during ingestion.	Slow query logs provide the primary input for identifying database performance issues.
FR-002	Functional	The system shall parse PostgreSQL/MySQL EXPLAIN plans and identify performance bottlenecks.	High	Pass: Sequential scans on large tables are identified correctly. Fail: A sequential scan is incorrectly marked as optimized.	Execution-plan analysis helps identify inefficient database operations.
FR-003	Functional	The system shall identify missing index candidates and recommend optimal index column definitions.	High	Pass: Missing composite indexes on filtered columns are identified. Fail: No index recommendation is generated for an eligible query.	Index recommendations can improve query execution performance.
FR-004	Functional	The system shall generate weekly performance optimization digests.	Medium	Pass: A weekly digest containing performance issues and recommendations is generated. Fail: The weekly digest is not generated.	Weekly summaries help stakeholders track recurring performance issues.
FR-005	Functional	The system shall allow authorized users to view query performance reports and recommendations.	Medium	Pass: Authorized users can view generated reports and recommendations. Fail: Authorized users cannot access the analysis.	Stakeholders need access to performance findings to take corrective action.
NFR-001	Performance & Security	Slow query log ingestion shall handle up to 5,000 log records per minute without dropping entries or slowing the host database.	High	Pass: 5,000 records/minute are processed without dropped entries or unacceptable database slowdown. Fail: Records are dropped or unacceptable slowdown occurs.	The profiler must support high-volume workloads without affecting the monitored database.
NFR-002	Reliability & Security	The system shall securely process database logs and execution plans while maintaining reliable operation during peak workloads.	High	Pass: Security and reliability tests succeed under simulated peak workload. Fail: Sensitive information is exposed or the profiler becomes unavailable.	Logs and execution plans may contain sensitive information and require secure, reliable processing.

3. UML Use-Case Diagram

Actors: Database Administrator and Backend Lead. Primary use cases cover log ingestion, execution-plan parsing, performance analysis, missing-index identification, and recommendation generation. Secondary use cases cover viewing reports and generating the weekly optimization digest.

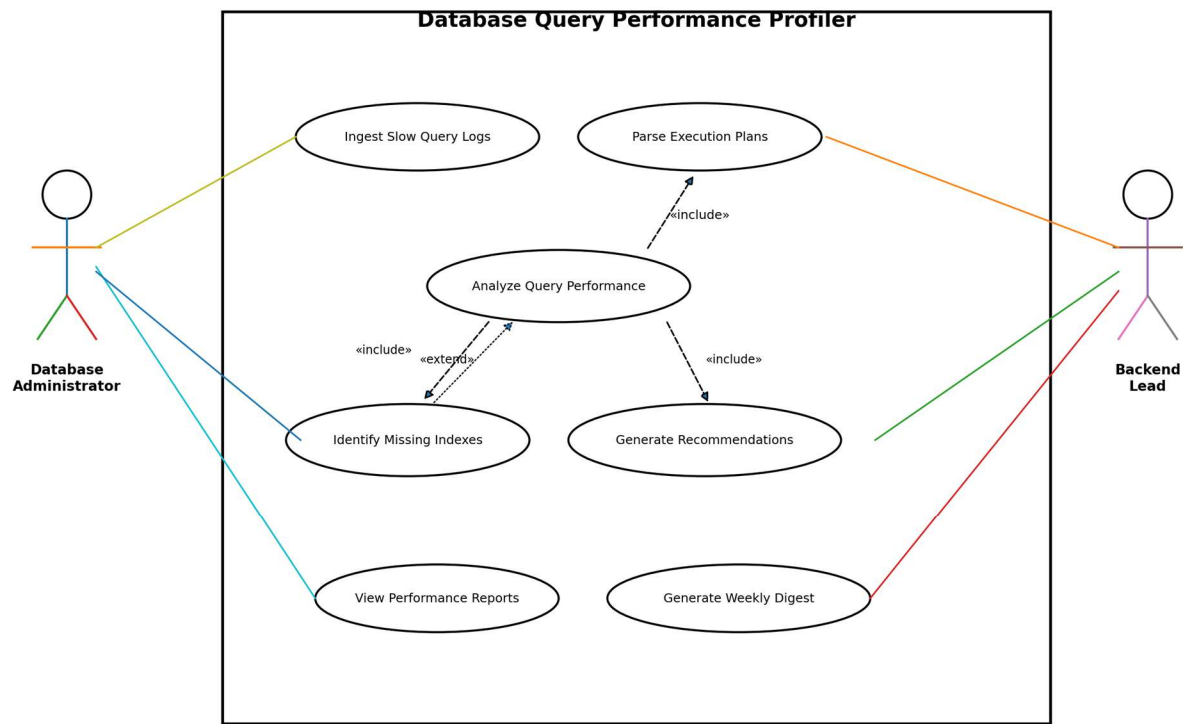


Figure 1: UML Use-Case Diagram

Relationship note: Analyze Query Performance «include» Parse Execution Plans and Generate Recommendations. Identify Missing Indexes is shown with an «extend» relationship to satisfy the required UML relationship constraint.

4. Main Success Scenario and Alternate Flow

Core Use Case: Analyze Query Performance

Primary Actor: Database Administrator

Preconditions: 1. Slow query logs have been ingested into the system. 2. A valid SQL execution plan is available for analysis.

Postconditions: 1. Query performance issues are identified. 2. Optimization recommendations are generated and made available to the user.

Main Success Scenario

1. Database Administrator selects a slow query for analysis.
2. The system retrieves the corresponding SQL execution plan.
3. The system parses the execution plan.
4. The system analyzes query operations and identifies performance bottlenecks.
5. The system checks for potential missing indexes.
6. The system generates optimization recommendations.
7. The system displays the analysis results and recommendations to the Database Administrator.

Alternate Flow

8. At Step 3, if the execution plan is invalid or cannot be parsed, the system reports an execution-plan parsing error.
9. The Database Administrator is prompted to provide a valid execution plan or select another query.
10. The system resumes the analysis when valid input is provided.

Submission checklist

- Save this as a single document.
- Verify that all 7 requirements and all 6 table fields are present.
- Verify that the UML diagram contains both actors and at least one «include» and one «extend».
- Verify that the use-case flow contains preconditions, postconditions, main success scenario, and one alternate flow.
- Upload the single document to the existing SE_LAB1_PES1UG24AM284 GitHub repository.
-